

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_c81dbe7a-7ff\agent-aa37b8f68c3130d16.jsonl`
- SHA-256: `032ac69d18f5a3c1be22f02b626cb5999013100069c0a215e444539b9e4fd9c1`
- Source modified: `2026-06-25T17:00:23+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:57:05.921Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [LOW] Grace boundary, multiple-stale, and timezone-offset cases are untested
Location: `tests/test_provider_gemini.py:119-163`
Detail: Coverage uses only 2h-stale and 5min-fresh fixtures. Untested: (1) the exact grace boundary ? prod uses strict ``created > cutoff`` (`gemini.py:76`), so a file created exactly at the cutoff IS deleted; no test pins this inequality. (2) Multiple stale videos deleted in one sweep (every happy-path test deletes at most one, so an off-by-one or early-return in the loop would not be caught ? the `per-file-delete-failure` test has two stale files but the first raises). (3) A `create_time` string carrying a non-Z numeric offset (e.g. `+05:30`) to prove the ``replace("Z",...)`` handling is offset-agnostic.
Suggested fix: Add: a fixture created at exactly ``now - grace_sec`` asserting it is deleted (or adjust to ``>=`` and document); a case with 2+ stale videos asserting `deleted==N` and two delete calls; and a `+05:30`-offset timestamp.

Return `isReal` + `adjustedSeverity` (`INVALID` if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:57:08.191Z)

I'll verify this finding by examining the actual code and tests in PR #380.

3. user (2026-06-25T16:57:09.872Z)

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
86705c1 chore: ruff format `gemini_refine.py` + `trajectory_hybrid.py` (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
---BRANCH---
o1-p11/gemini-remote-file-gc

4. user (2026-06-25T16:57:10.408Z)

```
backend/main.py          | 25 ++++++++
backend/providers/gemini.py | 43 +++++
tests/test_provider_gemini.py | 104 +++++
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:57:12.499Z)

Let me read the actual code and test files.

6. user (2026-06-25T16:57:13.649Z)

```
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
import logging
import os
import time
+from datetime import datetime, timedelta, timezone
from typing import Tuple

from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

+
+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+     return deleted
+
+
+ def analyze_video(self, video_path: str, prompt: str,
+                   chunk_duration: float = 0.0,
+                   label: str = "") -> Tuple[list, dict]:
```

7. user (2026-06-25T16:57:14.116Z)

```
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
     provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
     segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
     assert len(segments) == 0
+
+
+
+# --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),          # non-video -> kept
+        _file_mock("notts", "video/mp4", None),             # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+        assert deleted == 1
+        provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
+startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
```



```

50         Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
51         creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
52         Never raises; returns the count deleted. Intended to run once at startup.
53
54         Args:
55             grace_sec: only delete files older than this (default 1h). Protects a file a
56                 live worker may still be processing.
57         """
58         cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
59         deleted = 0
60         try:
61             files = list(self.client.files.list())
62         except Exception as e:
63             logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64
65         return 0
66     for f in files:
67         try:
68             mime = getattr(f, "mime_type", "") or ""
69             if not mime.startswith("video/"):
70                 continue
71             # f.create_time is an RFC3339 string on the File API.
72             ct = getattr(f, "create_time", None)
73             if ct is None:
74                 # No timestamp ? can't prove it's stale; leave it (safe default).
75                 continue
76             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
77             if created > cutoff:
78                 continue # too recent ? a live worker may own it
79             self.client.files.delete(name=f.name)
80             deleted += 1
81         except Exception as e:
82             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
83     if deleted:
84         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
85     return deleted

```

10. user (2026-06-25T16:57:18.378Z)

```

104
105
106
107     # --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone
122         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
124         files = [

```

```

125         _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
126         _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
127         _file_mock("old-img", "image/png", stale),          # non-video -> kept
128         _file_mock("notts", "video/mp4", None),            # no timestamp -> kept (safe)
129     ]
130     with patch("backend.providers.gemini.genai"):
131         provider = GeminiProvider(api_key="k", model_name="m")
132         provider._client = MagicMock()
133         provider._client.files.list.return_value = iter(files)
134         deleted = provider.cleanup_orphaned_files(grace_sec=3600)
135     assert deleted == 1
136     provider._client.files.delete.assert_called_once_with(name="old-vid")
137
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
141         startup."""
142         with patch("backend.providers.gemini.genai"):
143             provider = GeminiProvider(api_key="k", model_name="m")
144             provider._client = MagicMock()
145             provider._client.files.list.side_effect = RuntimeError("network")
146             assert provider.cleanup_orphaned_files() == 0
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),
154             _file_mock("b", "video/mp4", stale),
155         ]
156         with patch("backend.providers.gemini.genai"):
157             provider = GeminiProvider(api_key="k", model_name="m")
158             provider._client = MagicMock()
159             provider._client.files.list.return_value = iter(files)
160             # First delete raises, second succeeds ? _safe_delete swallows it.
161             provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163             assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []

```

```

189 monkeypatch.setattr(main_mod.provider_registry, "get",
190                       lambda *a, **k: called.append(k) or MagicMock())
191 main_mod._maybe_sweep_gemini_orphans(cfg)
192 assert called == []
193
194
195 def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196     """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
197     import backend.main as main_mod
198     cfg = MagicMock()
199     cfg.ai_provider = "gemini"
200     cfg.ai_model = "gemini-2.5-flash"
201     provider = MagicMock()
202     provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
203     monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
204     monkeypatch.setenv("GEMINI_API_KEY", "k")
205     # Must not raise.
206     main_mod._maybe_sweep_gemini_orphans(cfg)
207     provider.cleanup_orphaned_files.assert_called_once()

```

11. assistant (2026-06-25T16:57:32.050Z)

Now I have the full picture. Let me verify the three specific claims in the finding against the actual code.

****Claim 1 ? Grace boundary inequality (`created > cutoff`) untested.**** Confirmed: gemini.py:76 uses strict `created > cutoff`, so a file created exactly at the cutoff is NOT skipped ? it IS deleted. The tests use 2h-stale and 5min-fresh fixtures (lines 122-123) ? neither sits at the boundary. No test pins the inequality. True.

****Claim 2 ? Multiple stale videos in one sweep untested.**** Let me check each test's stale-video count:

- `test\_cleanup\_orphaned\_files\_deletes\_stale\_videos\_only` (line 119): exactly ONE stale video (`old-vid`); the other video is fresh, asserts `deleted == 1`.
- `test\_cleanup\_orphaned\_files\_per\_file\_delete\_failure\_is\_swallowed` (line 148): TWO stale videos, but the first delete RAISES, so only the second is counted (`deleted == 1`).

The finding's nuance is precise: no happy-path test deletes >1 file. An early-`return`/`break` after the first successful delete, or a loop that only processes one item, would still pass every test (the multi-file test only proves the loop continues **past a raised exception**, not that two successful deletes both increment). The `assert\_called\_once\_with` in the first test actually **pins** single-delete behavior, so a 2-success case is genuinely uncovered. True.

****Claim 3 ? Non-Z numeric offset (e.g. +05:30) untested.**** The fixtures use `.isoformat()` on timezone-aware UTC datetimes, which produces `+00:00` (not `Z`). So the `.replace("Z", "+00:00")` is effectively a no-op in tests, and a `+05:30` offset is never exercised. `datetime.fromisoformat` does handle offsets, so this is genuinely just a coverage gap, not a bug. True but the weakest of the three (the existing fixtures already prove offset parsing works for `+00:00`).

All three sub-claims are factually accurate. The code is correct; this is a pure test-coverage gap. The severity assessment: these are missing edge-case tests for a best-effort, non-critical startup GC. The boundary case (#1) is the most defensible since it pins an explicit inequality decision; the multiple-stale case (#2) is a real off-by-one/early-return blind spot; the offset case (#3) is marginal. This is appropriately LOW ? actually borderline NIT, but the multiple-stale gap (an early-return would ship undetected) gives it enough substance to stay at LOW.

12. user (2026-06-25T17:00:16.593Z)

Structured output provided successfully

13. assistant (2026-06-25T17:00:23.582Z)

I've completed my adversarial verification of the finding.

The finding is **real** with severity **LOW**. All three sub-claims in the finding are factually accurate against the actual code in PR #380:

- **Grace boundary**: ``backend/providers/gemini.py:76`` uses strict ``created > cutoff``, and no test exercises the exact-cutoff case.
- **Multiple stale videos**: The only happy-path delete test (``tests/test_provider_gemini.py:119-136``) deletes exactly one file and pins it with ``assert_called_once_with``; the two-stale-file test (lines 148-163) only proves the loop survives an exception, never that two *successful* deletes both increment. An early-return/break would ship undetected.
- **Timezone offset**: Fixtures use ``isoformat()`` on UTC-aware datetimes (emits ``+00:00``, never ``Z`` or a non-zero offset), so the ``replace("Z", ...)`` handling for ``+05:30`` is unexercised.

The underlying GC code is correct ? this is purely a test-coverage gap on a best-effort startup sweep, so LOW (not MEDIUM) is the right severity.